

PRACTICUM REPORT 11TH WEEK
PROGRAMMING ALGORITHM

Comparative Analysis of Programs With and Without Pointers



Lecturer:

Dimas Novian Aditia Syahputra, S. Tr. T., M. Tr. T.

Prepared by:

Name: Aisyah Currents

NIM (Student id): 25091397108

Class: International

UNIVERSITAS NEGERI SURABAYA
FACULTY OF VOCATION
INFORMATICS MANAGEMENT STUDY PROGRAM
YEAR 2025

TABLE OF CONTENTS

TABLE OF CONTENTS.....	1
CHAPTER I	2
INTRODUCTION.....	2
1.1. Background.....	2
1.2. Objective	2
CHAPTER II.....	3
THEORETICAL BASIS.....	3
2.1. Definition of Pointers.....	3
2.2. Memory Address and Dereferencing.....	3
CHAPTER III	4
METHODOLOGY	4
3.1. Tools and Materials	4
3.2. Implementation Method.....	4
CHAPTER IV	10
RESULTS AND DISCUSSION	10
4.1. Program Result.....	10
4.1.1. The Pointer-Based Program	10
4.1.2. The Non-Pointer Program	10
4.2. Performance Comparison	10
CHAPTER V	12
CLOSING.....	12
5.1. Conclusion.....	12

CHAPTER I

INTRODUCTION

1.1. Background

In the C and C++ programming languages, pointers are one of the most fundamental yet challenging concepts for beginners to understand. A pointer is a variable that stores the memory address of another variable, allowing direct manipulation of data in memory. Programs that utilize pointers often have more flexibility and efficiency in handling data, especially for operations involving arrays, dynamic memory allocation, and function arguments. However, programs without pointers are generally simpler and easier to read, but they may consume more memory or require additional computational steps. By comparing these two types of programs, students can better understand how pointers affect program structure, performance, and memory usage.

1.2. Objective

The objectives of this practicum are as follows:

1. To understand the basic concept and function of pointers in programming.
2. To identify the advantages and disadvantages of using pointers in real programming scenarios.
3. To enhance students' skills in writing and analyzing pointer-based code in C/C++

CHAPTER II

THEORETICAL BASIS

2.1. Definition of Pointers

A pointer is a special type of variable that stores the memory address of another variable instead of the actual data value. In C and C++ programming, pointers allow direct access and manipulation of memory, making them powerful tools for efficient data handling. The use of pointers enables programmers to create dynamic data structures such as linked lists, trees, and graphs, and to pass large amounts of data to functions without excessive memory copying.

2.2. Memory Address and Dereferencing

Every variable in a program is stored in a specific memory location, identified by its memory address. A pointer can hold this address, allowing programmers to directly access or modify the data stored at that location. The dereferencing operator (*) is used to access the value stored at the address a pointer refers to.

For example:

```
01. int x = 10;
02. int *ptr = &x; // pointer stores the address of x
03. cout << *ptr; // outputs 10 (the value of x)
04.
```

This mechanism allows greater control over data but also introduces risks if memory is accessed incorrectly, such as segmentation faults or undefined behavior.

CHAPTER III

METHODOLOGY

3.1. Tools and Materials

In conducting this practicum, several tools and materials were used to design, implement, and test both programs. The tools and materials are as follows:

1. Visual Studio Code
Used as the main Integrated Development Environment (IDE) for writing, editing, and debugging both C++ and Python programs.
2. C++ Compiler
Used to compile and execute the C++ program that implements pointer-based image processing.
3. Python 3 Interpreter
Used to run the Python version of the program that performs image extraction without pointers.
4. stb_image.h Library
Utilized in the C++ program to load and process image files in RGBA format.
5. Pillow Library
Used in Python to handle image loading and pixel data extraction.
6. CSV Module
Used to export pixel data into CSV format.
7. Test Image File
Image file used as the input for both programs during testing.
8. Laptop
Hardware device used for running and benchmarking both programs.

3.2. Implementation Method

The methodology in this practicum involves developing two programs with similar functionality but different programming approaches, one using pointers in C++ and another without pointers in Python. Both programs are designed to read an image, extract RGBA pixel values, and save the results into a CSV file. The following steps were carried out:

1. Designing the Algorithm
The overall algorithm is divided into three stages: image loading, pixel extraction, and CSV file export. Both versions follow the same logic to ensure fair comparison.
2. Implementing the Pointer-Based Program (C++)
The Source Code:

```

1.  #include <iostream>
2.  #include <fstream>
3.  #include <string>
4.  #include <chrono>
5.  #include <filesystem>
6.
7.  // Activate stb_image implementation in this translation unit.
8.  // stb_image is a lightweight header-only library for image loading.
9.  #define STB_IMAGE_IMPLEMENTATION
10. #include "stb_image.h"
11.
12. using namespace std;
13. using namespace std::chrono;
14. namespace fs = std::filesystem;
15.
16. /**
17.  * Load an image from disk and force it to RGBA format (4 channels).
18.  * @param filename  path to the image file
19.  * @param width     image width (output parameter)
20.  * @param height    image height (output parameter)
21.  * @param channels   number of original image channels before conversion
22.  * @return          pointer to the RGBA byte buffer (width * height * 4)
23.  *                 Must be freed later using stbi_image_free().
24.  */
25. unsigned char* loadImageRGBA(const string& filename, int& width, int& height, int& channels)
26. {
27.     // Load the image as RGBA (4 channels).
28.     unsigned char* data = stbi_load(filename.c_str(), &width, &height, &channels, 4);
29.     if (!data) {
30.         cerr << "Failed to load image: " << filename << endl;
31.         exit(1); // Stop program if loading fails.
32.     }
33.     cout << "Image loaded: " << filename << " (" << width << "x" << height << "), RGBA channels.\n";
34.     return data;
35. }
36. /**
37.  * Save RGBA pixel data into a CSV file with columns: x, y, R, G, B, A.
38.  * The iteration goes line-by-line (row by row).
39.  */
40. void saveToCSV(const unsigned char* data, int width, int height, const string& outputFile) {
41.     ofstream file(outputFile);
42.     if (!file.is_open()) {
43.         cerr << "Could not create output file: " << outputFile << endl;
44.         return;
45.     }
46.
47.     // Write the CSV header line

```

```

48.     file << "x,y,R,G,B,A\n";
49.
50.     // Pointer walks through the RGBA buffer sequentially
51.     const unsigned char* ptr = data;
52.
53.     // Nested loops: first iterate by y (rows), then by x (columns)
54.     for (int y = 0; y < height; ++y) {
55.         for (int x = 0; x < width; ++x) {
56.             // Read 4 consecutive bytes as R, G, B, A values
57.             int r = *(ptr++);
58.             int g = *(ptr++);
59.             int b = *(ptr++);
60.             int a = *(ptr++);
61.
62.             // Write one pixel's information to CSV
63.             file << x << "," << y << "," << r << "," << g << "," << b << "," << a << "\n";
64.         }
65.     }
66.
67.     file.close();
68.     cout << "RGBA data saved to: " << outputFile << endl;
69. }
70.
71. /**
72.  * MAIN ENTRY POINT – this is the part that is executed ("run").
73.  * - Determines current working directory
74.  * - Checks if the image file exists
75.  * - Loads the image as RGBA
76.  * - Exports pixel data to a CSV file
77.  * - Measures the total processing time
78.  */
79. int main() {
80.     auto start = high_resolution_clock::now(); // Start stopwatch
81.
82.     // Get the current working directory (where the executable runs)
83.     fs::path base = fs::current_path();
84.
85.     // Define paths for the input image and output CSV
86.     fs::path imagePath = base / "VynCompressed.png";
87.     fs::path outputCSV = base / "output.csv";
88.
89.     cout << "Current working directory: " << base.string() << "\n";
90.     cout << "Looking for image: " << imagePath.string() << "\n";
91.
92.     // Verify that the image file exists
93.     if (!fs::exists(imagePath)) {
94.         cerr << "File not found: " << imagePath.string() << "\n";
95.         return 1; // Exit with error code if image is missing
96.     }

```

```

97.
98.     int width = 0, height = 0, channels = 0;
99.
100.    // LOAD IMAGE → returns pointer to RGBA byte data
101.    unsigned char* imgData = loadImageRGBA(imagePath.string(), width, height, channels);
102.    if (!imgData) return 1; // Additional safety check
103.
104.    // SAVE PIXEL DATA TO CSV
105.    saveToCSV(imgData, width, height, outputCSV.string());
106.
107.    // FREE MEMORY allocated by stb_image
108.    stbi_image_free(imgData);
109.
110.    // Measure and print total execution time
111.    auto end = high_resolution_clock::now();
112.    chrono::duration<double> seconds = end - start;
113.    cout << "Processing time: " << seconds.count() << " seconds.\n";
114.
115.    return 0; // Program finished successfully
116. }

```

3. Implementing the Non-Pointer Program (Python)

The Source Code:

```

1.  from PIL import Image      # Pillow library to handle image operations (open, convert, get pixels)
2.  import csv                 # Built-in module for writing CSV files
3.  import time                # Module for measuring execution time (performance analysis)
4.
5.  def load_image(path):
6.      """Load an image from the given file path and convert it into RGBA format."""
7.      try:
8.          # Record the start time to measure image loading duration
9.          start_time = time.time()
10.
11.          # Open the image file and convert it to RGBA mode (Red, Green, Blue, Alpha)
12.          img = Image.open(path).convert("RGBA")
13.
14.          # Record the end time after loading the image
15.          end_time = time.time()
16.
17.          # Display information about the image
18.          print(f"Image loaded: {path}")
19.          print(f"Image size: {img.width}x{img.height} pixels")
20.          print(f"Image load time: {end_time - start_time:.4f} seconds")
21.
22.          # Return the loaded image object for further processing
23.          return img
24.

```



```

25.     except FileNotFoundError:
26.         # Handle case where the file path is invalid or missing
27.         print("Image not found.")
28.         return None
29.
30.
31. def save_rgba_to_csv(img, output_csv):
32.     """Save all pixel coordinates (x, y) and RGBA values to a CSV file."""
33.     if img is None:
34.         # If no image is loaded, skip processing
35.         print("No image to process.")
36.         return
37.
38.     # Record start time before writing to CSV
39.     start_time = time.time()
40.
41.     # Open (or create) a CSV file in write mode
42.     with open(output_csv, mode="w", newline="") as file:
43.         writer = csv.writer(file)
44.
45.         # Write CSV header row (column names)
46.         writer.writerow(["x", "y", "R", "G", "B", "A"])
47.
48.         # Loop through each pixel row (y-coordinate)
49.         for y in range(img.height):
50.             # Loop through each pixel column (x-coordinate)
51.             for x in range(img.width):
52.                 # Extract the RGBA value of the current pixel
53.                 r, g, b, a = img.getpixel((x, y))
54.
55.                 # Write the pixel coordinate and RGBA values into one CSV row
56.                 writer.writerow([x, y, r, g, b, a])
57.
58.                 # Optional: Uncomment below to print pixel data to console
59.                 # print(f"({x}, {y}) → R:{r}, G:{g}, B:{b}, A:{a}")
60.
61.     # Record end time after processing
62.     end_time = time.time()
63.
64.     # Display completion messages
65.     print(f"\nRGBA data successfully saved to: {output_csv}")
66.     print(f"Processing time: {end_time - start_time:.4f} seconds")
67.
68.
69. def main():
70.     """Main function to coordinate image loading and CSV export."""
71.     # Start overall execution timer
72.     total_start = time.time()
73.

```

```

74.     # Ask the user for image input path and output CSV filename
75.     path = input("Enter image filename: ").strip()
76.     output_csv = input("Enter output CSV filename: ").strip()
77.
78.     # Load image using the load_image() function
79.     img = load_image(path)
80.
81.     # If loading successful, proceed to save RGBA data into CSV
82.     if img:
83.         save_rgba_to_csv(img, output_csv)
84.
85.     # Record the total end time
86.     total_end = time.time()
87.
88.     # Display total program execution time (loading + saving)
89.     print(f"\nTotal execution time: {total_end - total_start:.4f} seconds")
90.
91.
92. # Run the program only when executed directly (not when imported as a module)
93. if __name__ == "__main__":
94.     main()

```

4. Performance Comparison

Both programs are tested using the same input image and hardware conditions. Then total execution time and output CSV size are compared to determine which approach is faster and more efficient.

CHAPTER IV

RESULTS AND DISCUSSION

4.1. Program Result

4.1.1. The Pointer-Based Program

This output comes from the C++ version of the program that uses pointers to process image data directly from memory.

```
C:\pointer>rgba_extract.exe
Current working directory: C:\pointer
Looking for image: C:\pointer\VynCompressed.png
Image loaded: C:\pointer\VynCompressed.png (736x436), RGBA channels.
RGBA data saved to: C:\pointer\output.csv
Processing time: 1.80814 seconds.
```

This output demonstrates that the pointer-based program efficiently loads and converts image data into CSV format in a relatively short time.

4.1.2. The Non-Pointer Program

This output comes from the Python implementation, which does not use pointers. Instead, it uses high-level functions from the PIL library.

```
Enter image filename: VynCompressed.png
Enter output CSV filename: Vyn.csv
Image loaded: VynCompressed.png
Image size: 736x436 pixels
Image load time: 0.2853 seconds

RGBA data successfully saved to: Vyn.csv
Processing time: 3.2569 seconds
```

The non-pointer version is easier to write and understand, but slower because it uses higher-level memory management and loops handled by the Python interpreter instead of direct memory access.

4.2. Performance Comparison

The performance comparison between the pointer-based (C++) and non-pointer (Python) programs aims to evaluate their efficiency in processing image data and exporting it into CSV format. Both programs perform the same task using the same image file, allowing a fair measurement of execution time and resource utilization. The table below summarizes the results, highlighting differences in speed, memory access, and overall implementation characteristics.

Criteria	Pointer-Based Program (C++)	Non-Pointer Program (Python)	Remarks / Analysis
Programming Language	C++	Python	C++ is compiled and closer to hardware; Python is interpreted and higher-level.
Memory Access Method	Uses pointers for direct memory access	Uses variable references and managed memory	Pointer access allows faster read/write operations.
Image File Tested	VynCompressed.png (736×436 pixels)	VynCompressed.png (736×436 pixels)	Both programs use the same input for fairness.
Image Loading Time	~0.28 seconds	~0.29 seconds	Both load images at similar speed due to library optimization.
Data Conversion and CSV Writing Time	~1.80 seconds (total)	~3.26 seconds (total)	The pointer-based version is about 1.8× faster.
Output File	output.csv	Vyn.csv	Both successfully generate CSV files with RGBA data.
Ease of Implementation	More complex (manual memory handling)	Easier and more readable	Python is simpler but less efficient.
Performance Efficiency	High (faster execution and lower overhead)	Moderate (slower due to interpreter overhead)	C++ performs better for large-scale pixel operations.

Based on the table above, the pointer-based C++ program demonstrates better performance, completing the image extraction and CSV export in approximately **1.8 seconds**, compared to the **3.26 seconds** required by the Python version. Although the Python program offers simplicity and readability, the pointer-based C++ approach achieves higher execution speed due to direct memory manipulation, which reduces data handling overhead. Therefore, for tasks involving large datasets or pixel-level processing, C++ with pointers provides better performance and efficiency.

CHAPTER V

CLOSING

5.1. Conclusion

Based on the practicum conducted on comparing programs that use pointers and those that do not, it can be concluded that the use of pointers has a significant impact on program performance and efficiency. The C++ program, which utilizes pointers, demonstrated faster execution time and more efficient memory usage when processing image data and converting it into a CSV file. This efficiency is achieved because pointers allow direct access to memory locations, reducing overhead during data manipulation.

In contrast, the Python program, which does not use pointers, operates at a higher abstraction level that prioritizes simplicity and safety but results in slower performance due to additional layers of memory management and object handling. Despite this, Python remains advantageous in terms of readability, development speed, and error handling.

Therefore, the choice between using pointers or not depends on the specific needs of the program. Whether it prioritizes performance and control (as in C++) or simplicity and rapid development (as in Python).